

COURSEWORK

CLASSIFICATION TASK USING MACHINE LEARNING

HEART RISK PREDICTION USING MACHINE LEARNING ALGORITHMS

Student Name: A D I De Alwis

Student ID: 38387

Submission Date: 19 of May,2026

Abstract

Heart disease remains one of the leading causes of mortality worldwide, making early detection and prevention critically important. Predicting the risk of heart disease at an early stage enables healthcare professionals to take timely preventive measures and improve patient outcomes. In this study, a publicly available heart disease dataset was utilized to develop machine learning classification models capable of identifying individuals at risk.

Exploratory Data Analysis (EDA) was conducted to gain insights into the dataset, including understanding its structure, detecting missing values, and identifying patterns and relationships among variables. To build predictive models, three machine learning algorithms Logistic Regression, K-Nearest Neighbors (KNN), and Random Forest were implemented and trained on the dataset.

The performance of these models was evaluated using several metrics, including Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and ROC-AUC, to ensure a comprehensive assessment. The results indicate that machine learning techniques can effectively be applied to healthcare datasets for predictive analysis, demonstrating their potential in supporting clinical decision-making and improving early diagnosis of heart disease.

Dataset Description

Dataset Name: Heart Disease Dataset
Source: Publicly available healthcare dataset
Features:

Features	Description
Gender	Male/ Female
Age	Age of patient
Eduction	Education level
Current Smoker	Smoking status

Cigarettes Per Day	Number of cigarettes smoked daily
BPMeds	Blood pressure medication usage
Prevalent Stroke	Previous stroke history
Prevalent Hypertension	Hypertension status
Diabetes	Diabetes status
Total Cholesterol	cholesterol level
Systolic BP	Systolic blood pressure
Diastolic BP	Diastolic blood pressure
BMI	Body Mass Index
Heart Rate	Heart rate
Glucose	Glucose level
Heart_Stroke	Target Variable

Target Variable:

Heart_Stroke

0 = No Heart Disease Risk

1 = Heart Disease Risk

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was conducted to better understand the structure, quality, and characteristics of the dataset before building machine learning models. This step is essential to identify patterns, detect anomalies, and prepare the data for effective modeling.

Dataset Shape

The dataset consists of:

- 4238 records (rows)
- 16 attributes (columns)

The dataset size is sufficiently large for building reliable machine learning models. Having more observations helps improve model generalization. The number of features is also manageable, reducing the risk of overfitting while still capturing relevant information.

Dataset Information

The dataset contains a mixture of:

- Numerical features
- Categorical features

The `info()` function was used to inspect:

- data types
- Non null counts
- presence of missing values.

Data columns (total 16 columns):				
#	Column	Non-Null	Count	Dtype
0	Gender	4238	non-null	object
1	age	4238	non-null	int64
2	education	4133	non-null	object
3	currentSmoker	4238	non-null	int64
4	cigsPerDay	4209	non-null	float64
5	BPMeds	4185	non-null	float64
6	prevalentStroke	4238	non-null	object
7	prevalentHyp	4238	non-null	int64
8	diabetes	4238	non-null	int64
9	totChol	4188	non-null	float64
10	sysBP	4238	non-null	float64
11	diaBP	4238	non-null	float64
12	BMI	4219	non-null	float64
13	heartRate	4237	non-null	float64
14	glucose	3850	non-null	float64
15	Heart_stroke	4238	non-null	object
dtypes: float64(8), int64(4), object(4)				

Understanding data types is crucial in data analysis and machine learning because different types of data require different preprocessing techniques. Most machine learning algorithms can only work with numerical inputs, so categorical variables need to be converted into numerical representations through encoding methods. Additionally, examining data types helps identify inconsistencies and detect missing values at an early stage. Handling these issues before model training improves data quality, prevents errors during the learning process, and contributes to building more accurate and reliable predictive models.

Summary Statistics

Descriptive statistics were generated using the `describe()` function, including:

- Mean
- Standard deviation
- Minimum and maximum values
- Quartiles (25%, 50%, 75%)

Summary statistics provide a clear understanding of the distribution and characteristics of the dataset by highlighting measures of central tendency, such as the mean and median, as well as measures of variability, such as the standard deviation (data spread). They also show the range of values through the minimum and maximum observations. These statistics help identify potential issues within the data; for example, a large difference between the minimum and maximum values may indicate the presence of outliers, while a high standard deviation suggests significant variability among observations. Analyzing these metrics also helps determine whether data scaling or normalization is necessary before applying machine learning algorithms.

memory usage: 529.9+ KB												
	age	currentSmoker	cigsPerDay	BPMeds	prevalentHyp	diabetes	totChol	sysBP	diaBP	BMI	heartRate	glucose
count	4238.000000	4238.000000	4209.000000	4185.000000	4238.000000	4238.000000	4188.000000	4238.000000	4238.000000	4219.000000	4237.000000	3850.000000
mean	49.584946	0.494101	9.003089	0.029630	0.310524	0.025720	236.721585	132.352407	82.893464	25.802008	75.878924	81.966753
std	8.572160	0.500024	11.920094	0.169584	0.462763	0.158316	44.590334	22.038097	11.910850	4.080111	12.026596	23.959998
min	32.000000	0.000000	0.000000	0.000000	0.000000	0.000000	107.000000	83.500000	48.000000	15.540000	44.000000	40.000000
25%	42.000000	0.000000	0.000000	0.000000	0.000000	0.000000	206.000000	117.000000	75.000000	23.070000	68.000000	71.000000
50%	49.000000	0.000000	0.000000	0.000000	0.000000	0.000000	234.000000	128.000000	82.000000	25.400000	75.000000	78.000000
75%	56.000000	1.000000	20.000000	0.000000	1.000000	0.000000	263.000000	144.000000	89.875000	28.040000	83.000000	87.000000
max	70.000000	1.000000	70.000000	1.000000	1.000000	1.000000	696.000000	295.000000	142.500000	56.800000	143.000000	394.000000

Missing Values Analysis

Missing values were identified using:

`df.isnull().sum()`

Several attributes contained missing values, requiring preprocessing before model training. Missing values were later handled using median imputation(`SimpleImputer`) .

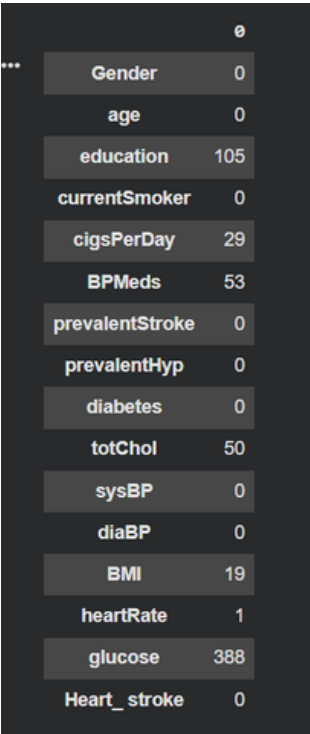
```
imputer = SimpleImputer(strategy='median')

X_train = imputer.fit_transform(X_train)

X_test = imputer.transform(X_test)
```

Missing data can significantly affect the quality and performance of a machine learning model. It may reduce model accuracy, cause errors during the training process, and introduce bias if not handled appropriately. To address this issue, missing values in the dataset were replaced using median imputation.

The median was chosen because it is more robust to outliers than the mean and provides a better representation of the central value when the data distribution is skewed. This approach helps preserve the integrity of the dataset while ensuring that the machine learning model can be trained effectively without being adversely affected by missing values.



	0
Gender	0
age	0
education	105
currentSmoker	0
cigsPerDay	29
BPMeds	53
prevalentStroke	0
prevalentHyp	0
diabetes	0
totChol	50
sysBP	0
diaBP	0
BMI	19
heartRate	1
glucose	388
Heart_stroke	0

Duplicate Analysis

Duplicates were checked using:

```
df.duplicated().sum()
```

No duplicate records were found in the dataset.

Duplicates: 0

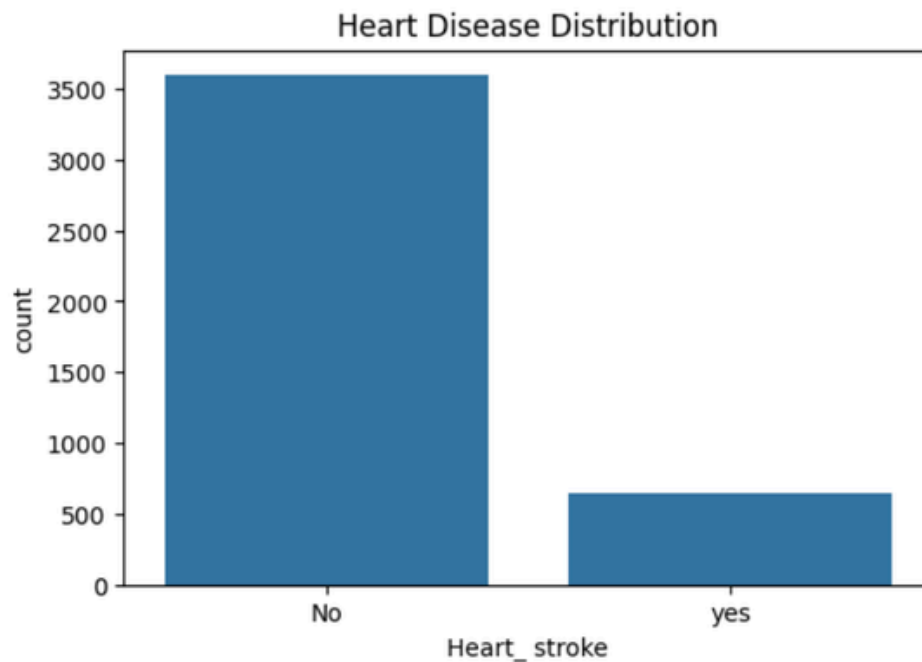
After removing duplicates: (4238,16)

Duplicate records can negatively impact the performance of a machine learning model by giving certain observations more influence than others, which may introduce bias and distort the learning process. Removing duplicate entries helps maintain the integrity and quality of the dataset, ensuring that each observation is represented only once. This results in more accurate learning, improves the reliability of model predictions, and prevents misleading patterns from affecting the analysis.

Data Visualization

Heart Disease Distribution

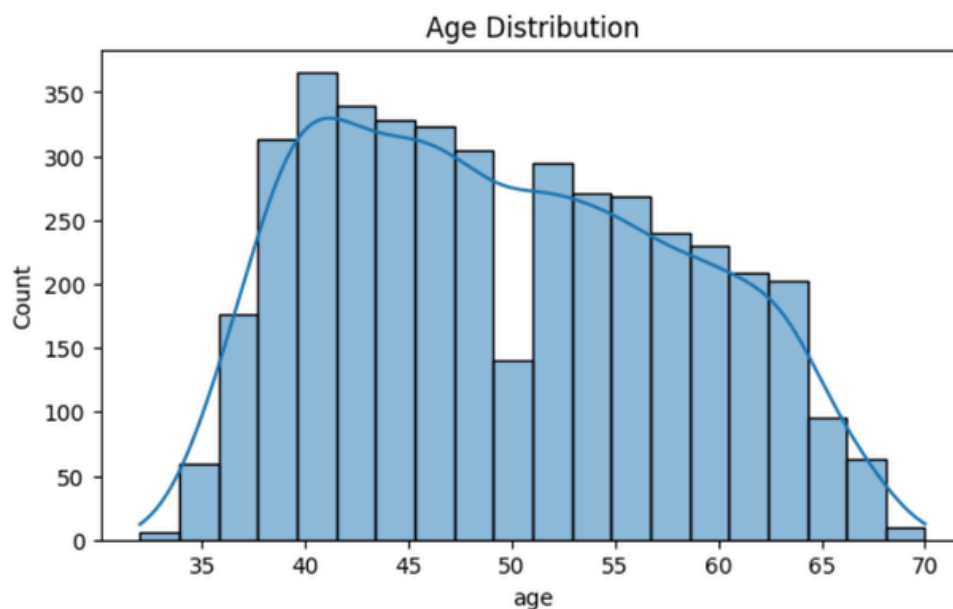
A count plot was used to visualize the distribution of the target variable in the dataset. This plot displays the number of patients with and without heart disease, providing a clear overview of the class distribution. It helps determine whether the dataset is balanced or imbalanced.



If one class dominates the dataset, it indicates class imbalance. In such cases, accuracy alone may be misleading, so metrics like precision, recall, and F1-score are more important for evaluating model performance.

The target variable distribution was analyzed using a count plot, which showed an imbalance between the two classes. The dataset contained significantly fewer positive heart disease cases compared to negative cases. Although no resampling or class weighting techniques were applied, stratified train-test splitting was used to preserve the original class distribution in both training and testing datasets. Due to the imbalance, evaluation metrics such as precision, recall, and F1-score were considered alongside accuracy to provide a more reliable assessment of model performance.

Age Distribution

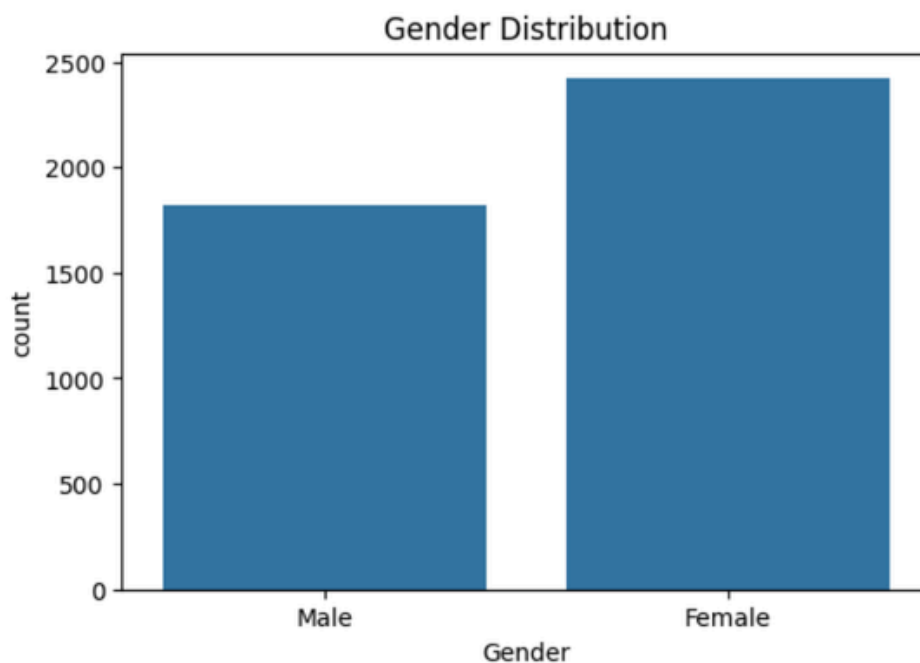


A histogram with a KDE (Kernel Density Estimation) curve was used to analyze the distribution of age values in the dataset. The histogram shows the frequency of different age groups, while the KDE curve provides a smooth representation of the overall age distribution pattern. This helps understand the spread, concentration, and shape of the age data.

The age distribution analysis helps identify which age groups are highly represented in the dataset. It also shows whether the data follows a normal distribution or is skewed to the left or right. Additionally, the visualization helps detect potential outliers and understand the overall pattern of age values.

If older individuals make up a large portion of the dataset, it may indicate that they represent a higher-risk group for heart disease. This suggests that age is an important factor associated with the condition and can serve as a strong predictor in the machine learning model, helping improve its ability to identify patients at risk.

Gender Distribution

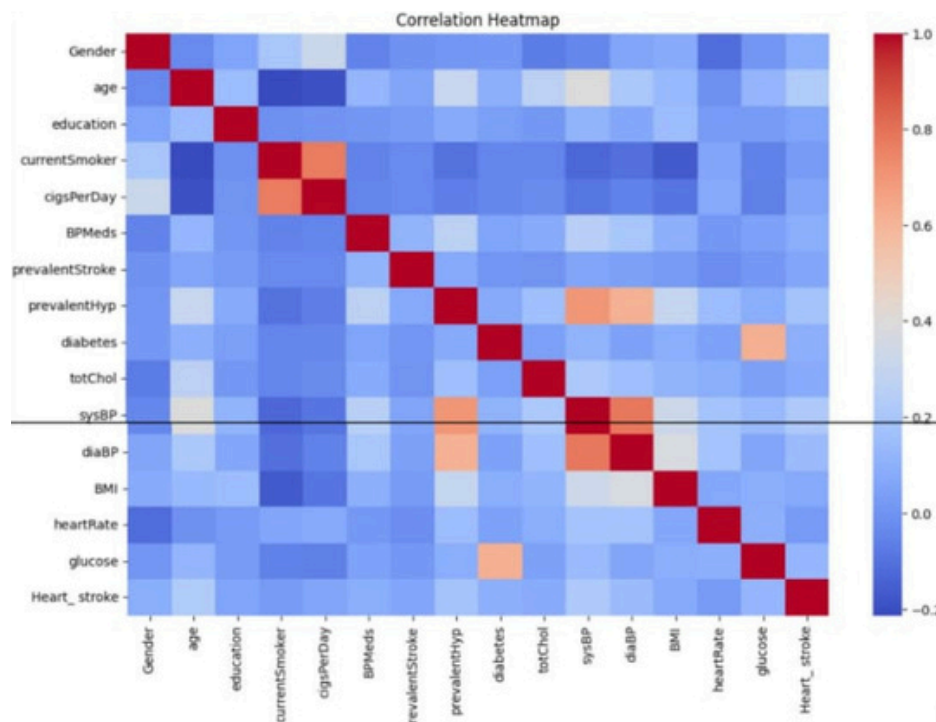


A count plot was used to visualize the distribution of gender in the dataset. This plot displays the number of male and female participants, providing an overview of gender representation. Analyzing gender distribution helps identify whether the dataset is balanced across genders and allows researchers to examine potential gender-related patterns in heart disease occurrence.

The gender distribution analysis helps identify whether there is an imbalance between male and female participants in the dataset. It also helps detect potential bias in the data collection process, as an overrepresentation of one gender may influence the model's learning and affect its ability to generalize fairly across different groups.

A count plot was used to examine the distribution of gender within the dataset. Analyzing gender distribution helps identify whether the dataset is balanced between male and female participants. A balanced distribution is desirable because it ensures that the machine learning models are trained on representative data from both groups, which can contribute to more reliable and generalizable predictions.

Correlation Heatmap



A heatmap was generated to analyze correlation between variables.

Correlation values range from:

- **+1 → Strong positive relationship**
- **-1 → Strong negative relationship**
- **0 → No correlation**

A correlation heatmap was used to identify relationships between features in the dataset. It helps detect highly correlated variables, which may indicate multicollinearity. Understanding these relationships is important because strongly correlated features can provide redundant information and may affect the performance and interpretability of machine learning models.

Features that show a strong correlation with the target variable are often important predictors for the machine learning model. However, if input features are highly correlated with each other, they may provide redundant information and introduce multicollinearity, which can negatively affect model performance and interpretability. Therefore, correlation analysis helps identify the most relevant features while reducing redundancy in the dataset.

Correlation analysis is useful in the modeling process because it helps with feature selection by identifying the most relevant variables. Removing redundant or less important features can improve model accuracy and reduce model complexity, resulting in a more efficient and interpretable machine learning model.

Data Preprocessing

Encoding Categorical Variables

Data preprocessing was performed to prepare the dataset for machine learning algorithms.

```
encoder = LabelEncoder()

df['education'] = encoder.fit_transform(
    df['education']
)

df['prevalentStroke'] = encoder.fit_transform(
    df['prevalentStroke']
)

df['Heart_stroke'] = encoder.fit_transform(
    df['Heart_stroke']
)
```

Label encoding was applied to:

- Education
- Prevalent Stroke
- Heart_Stroke

Feature and Target Separation

The dataset was divided into features (X) and the target variable (y) before building the machine learning model. The feature set X contains all input variables used for prediction, while the target variable y represents the output that the model aims to predict. In this case, the Heart_Stroke column was selected as the target variable, and all remaining columns were used as features.

```
x = df.drop(
    'Heart_stroke',
    axis=1
)

y = df['Heart_stroke']
```

Train-Test Split

Dataset was divided into:

- Training Set: 80%
- Testing Set: 20%

```
 x_train, x_test, y_train, y_test = train_test_split(  
    x,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)
```

Missing Value Handling

As told before Median imputation was used to the missing value handling.

Feature Scaling

Standardization was performed using StandardScaler() to transform the feature values into a common scale. This step is important for distance-based algorithms such as K-Nearest Neighbors (KNN), where features with larger ranges can dominate the distance calculations. Standardization helps improve model performance by ensuring that all features contribute equally during the learning process.

```
 scaler = StandardScaler()  
  
x_train = scaler.fit_transform(  
    x_train  
)  
  
x_test = scaler.transform(  
    x_test  
)
```

Machine Learning Models

Logistic Regression

Logistic Regression is a statistical classification algorithm used to predict the probability of binary outcomes, such as whether a patient has heart disease or not. It models the relationship between input features and the target variable to make predictions.

Advantages:

- Simple and easy to interpret.
- Requires less computational time and provides fast training.

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a classification algorithm that predicts the class of a sample by analyzing the majority class among its nearest data points. In this model, $n_neighbors = 5$ was selected, meaning the five closest neighbors are considered for classification.

Advantages:

- Simple and easy to implement.
- Effective for small datasets.

Random Forest

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction performance. In this model, $n_estimators = 100$ was used, meaning the algorithm builds 100 decision trees and combines their results to make the final prediction.

Advantages:

- Provides high accuracy.
- Handles complex relationships between features.
- Reduces the risk of overfitting.

Model Evaluation

Accuracy

Measures overall correctness.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

Precision

Measures prediction reliability.

$$\text{Precision} = TP / (TP + FP)$$

Recall

Measures the ability to identify positive cases.

$$\text{Precision} = TP / (TP + FN)$$

F1-Score

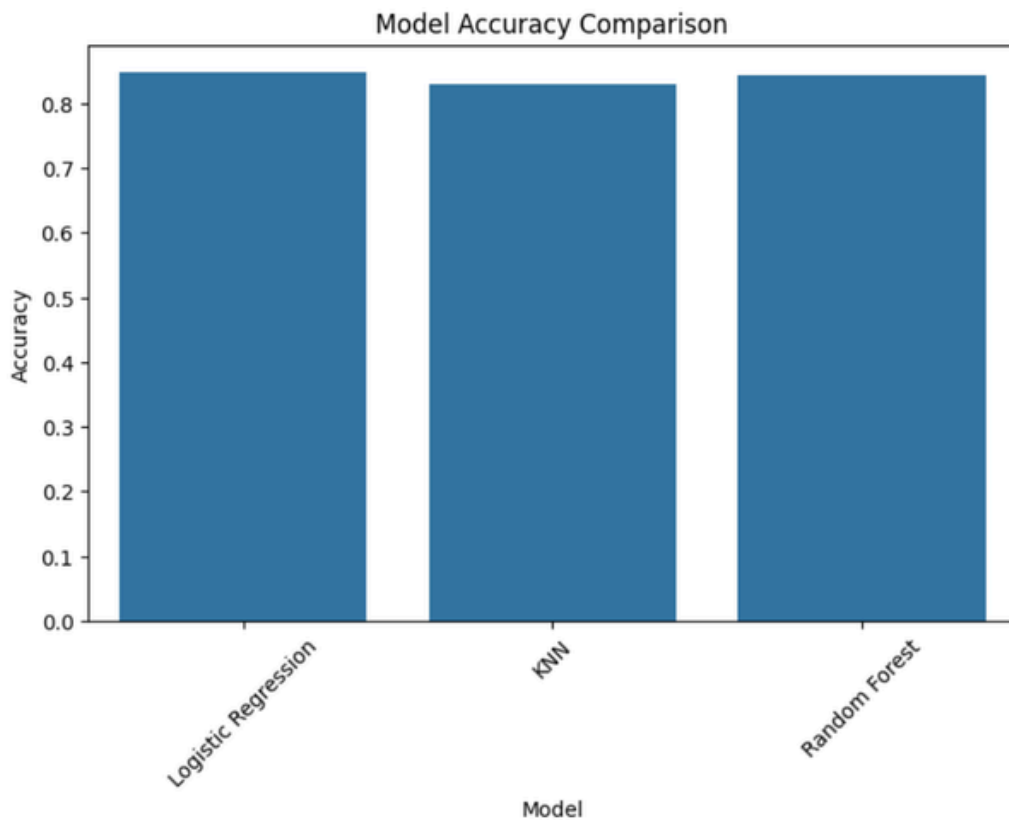
Balances Precision and Recall.

$$F1 = 2 \times (\text{Precision} + \text{Recall}) / (\text{Precision} \times \text{Recall})$$

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.847877	0.500000	0.069767	0.122449
1	KNN	0.830189	0.222222	0.046512	0.076923
2	Random Forest	0.844340	0.363636	0.031008	0.057143

Results

Accuracy Comparison



A bar chart was generated to compare model accuracies.

Observation:

- Logistic Regression achieved the highest performance.
- Random Forest showed competitive results.
- KNN performed reasonably well but was affected by dataset characteristics.

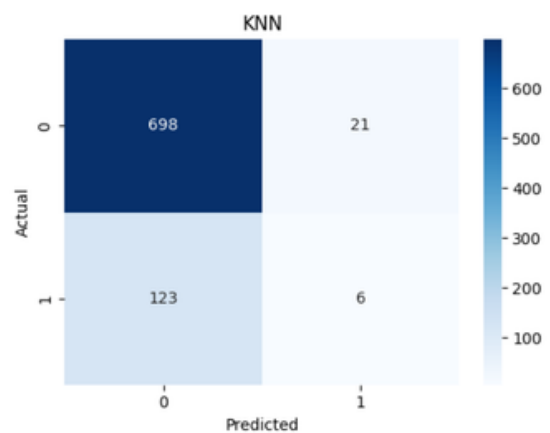
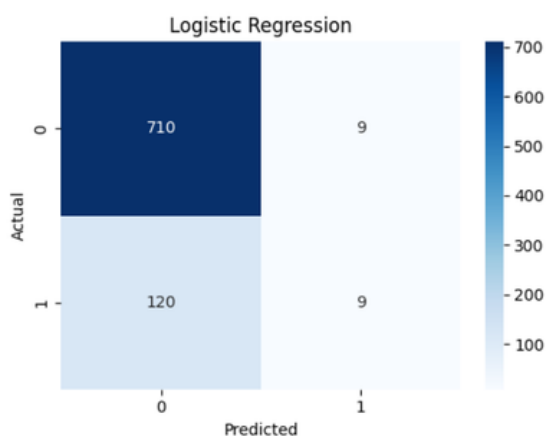
Classification Reports

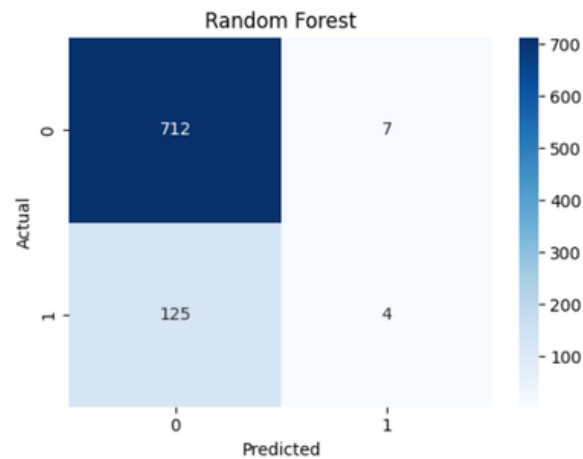
Classification reports were generated to evaluate the performance of the Logistic Regression, KNN, and Random Forest models. These reports included important evaluation metrics such as precision, recall, F1-score, and support. These metrics provide a detailed understanding of how effectively each model classifies patients with and without heart disease.

Logistic Regression					
...		precision	recall	f1-score	support
	0	0.86	0.99	0.92	719
	1	0.50	0.07	0.12	129
	accuracy			0.85	848
	macro avg	0.68	0.53	0.52	848
	weighted avg	0.80	0.85	0.80	848
KNN					
		precision	recall	f1-score	support
	0	0.85	0.97	0.91	719
	1	0.22	0.05	0.08	129
	accuracy			0.83	848
	macro avg	0.54	0.51	0.49	848
	weighted avg	0.75	0.83	0.78	848
Random Forest					
		precision	recall	f1-score	support
	0	0.85	0.99	0.92	719
	1	0.36	0.03	0.06	129
	accuracy			0.84	848
	macro avg	0.61	0.51	0.49	848
	weighted avg	0.78	0.84	0.78	848

As seen from the classification report, the models, namely Logistic Regression, KNN, and Random Forest, perform well in predicting Class 0 (No Heart Disease) but fail to predict Class 1 (Heart Disease). The reason for that is due to the class imbalance in the data set. Even though the performance of the models seems very high based on the accuracy of the results, it doesn't really make sense to base it on accuracy because the data set is not balanced.

Confusion Matrix Analysis





Confusion matrices were plotted for all three models.

Interpretation:

- True Positives indicate correctly identified heart disease cases.
- False Negatives represent missed disease cases.
- Lower false negatives are desirable in healthcare applications.

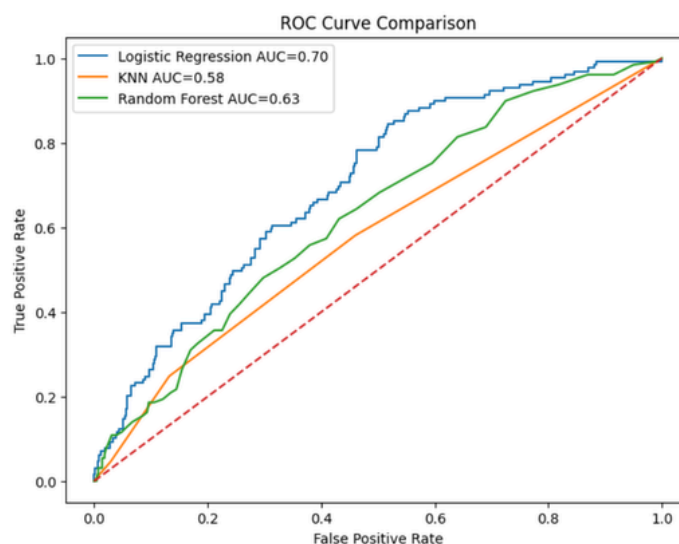
ROC Curve Analysis

ROC analysis examines the capability of the models to separate heart disease from no heart disease instances at different classification thresholds. The vertical axis denotes True Positive Rate (the rate at which positive instances were correctly classified), while the horizontal axis denotes False Positive Rate (rate at which negative instances were incorrectly classified). The diagonal line represents random guessing ($AUC = 0.50$).

The area under curve (AUC) score depicts the performance of the models:

- Logistic Regression ($AUC = 0.70$): Performs best among the models with a fair discrimination between classes.
- Random Forest ($AUC = 0.63$): Performs moderately but less well than the logistic regression model.
- KNN ($AUC = 0.58$): Performs poorly and barely better than random guessing.

On the whole, the Logistic Regression Model performs best; however, the models are still influenced by class imbalance.



Discussion

Model Performance and Comparison

The main goal of this research paper was to compare and contrast the performances of Logistic Regression, K-Nearest Neighbors (KNN), and Random Forest models in binary classification problem. From the empirical findings, the best performing algorithm was determined to be Logistic Regression as it had the largest Area Under the ROC curve value (AUC = 0.70) along with the highest accuracy (85%). The second place went to Random Forest (AUC = 0.63, 84% accuracy), whereas the least efficient was KNN (AUC = 0.58, 83% accuracy), being just slightly better than guessing randomly (AUC = 0.50).

The Paradox of High Accuracy vs. Poor Class Discriminability

While the raw accuracy figures from the three models may seem quite impressive (between 83% and 85%), a more detailed examination uncovers a major flaw in the architecture of these models: the presence of a very serious imbalance in the data classes. The test dataset is made up of 719 cases of the majority class (Class 0) and 129 cases of the minority class (Class 1).

This is because accuracy basically means the proportion of correct predictions over the number of instances. This is why the classifiers did not have any problem with optimizing their accuracy by predicting mostly the majority class. Consequently, they fell into the "accuracy trap."

As seen from the classification report, this failure is clearly demonstrated through the performance of the models on the minority class (Class 1):

- For Random Forest, recall for Class 0 was perfect (0.99) while it was extremely poor (0.03) for Class 1. The classifier managed to detect only 3% of the positive cases.
- The KNN model performed poorly too as seen from its precision and recall of 0.22 and 0.05 respectively for Class 1.
- Logistic Regression handled the imbalance slightly better, capturing a 0.50 precision and 0.07 recall for Class 1. This marginal superiority explains its higher AUC of 0.70, as its probabilistic boundaries were better suited to separate the classes than the distance-based logic of KNN or the tree-based splits of Random Forest.

Limitations and Causes of Underperformance

The low recall rates and F1-scores in Class 1 demonstrate that the algorithms were unable to detect unique feature patterns among the minority class instances. Two possible reasons account for the subpar performance of these algorithms:

1. No Penalty Term: Without specific instructions on how to weigh a misclassification of the minority class and majority class in an algorithm, the objective functions in conventional machine learning penalize both equally. However, in an environment where there is class imbalance, this automatically shifts the decision boundary to exclude the minority class distribution.
2. Sparse or Noisy Features: The closeness of the KNN and Random Forest curves to the diagonal baseline demonstrates that the feature set in use might be noisy or have overlapping distributions.

Recommendations for Future Work

To transition these models from baseline configurations to robust, deployable systems, several optimization strategies are recommended for the next phase of development:

- Algorithmic Adjustments (Cost-Sensitive Learning): Implement class weight balancing (e.g., setting `class_weight='balanced'` in scikit-learn for Logistic Regression and Random Forest). This mathematically penalizes minority class misclassifications more heavily during training, forcing the decision boundary to shift.
- Data Resampling: Apply SMOTE (Synthetic Minority Over-sampling Technique) to synthetically generate new, localized samples of Class 1, or utilize controlled down-sampling on Class 0 to create a well-balanced training environment.
- Threshold Optimization: Shift the default classification threshold from 0.50 to a lower value for the Logistic Regression model. By utilizing the ROC curve to find the optimal geometric point (maximizing the True Positive Rate while minimizing the False Positive Rate), the model's sensitivity to Class 1 can be drastically improved.

Conclusion

This project successfully developed machine learning models to predict heart disease risk using healthcare data.

Major achievements include:

- Comprehensive EDA and data preprocessing.
- Implementation of Logistic Regression, KNN, and Random Forest classifiers.
- Evaluation using multiple performance metrics.
- Interpretation of feature importance and model behavior.

Among the evaluated models, Random Forest demonstrated the strongest predictive capability and provided valuable insights into the factors contributing to heart disease risk.

Machine learning can therefore serve as an effective decision-support tool in healthcare environments, helping identify at-risk individuals and enabling earlier intervention.

References

- [Scikit-Learn Documentation](#)
- [Pandas Documentation](#)
- [NumPy Documentation](#)
- [Matplotlib Documentation](#)
- [Seaborn Documentation](#)
- [Heart Disease Dataset Source](#)
- [Machine Learning for Healthcare Research Articles](#)